An Independent Verification Framework for Reproducible TinyML Evaluation: From Model Artifacts to Deployment Claims

Narendra Satish

Abstract—The deployment of deep learning models on resource-constrained microcontrollers (TinyML) is rapidly expanding, yet empirical evaluation in the field suffers from critical methodological vulnerabilities. Training-time in-memory metrics frequently diverge from disk-serialized binary behavior; theoretical weight sparsity is routinely conflated with physical storage compression; and host simulation timings are improperly extrapolated to microcontroller real-time guarantees. In this paper, we develop and evaluate an empirical verification protocol for compiled TinyML artifacts, demonstrated on an end-to-end TinyML diagnostic case study. The protocol formalizes verification procedures across seven core dimensions: Data Isolation, Serialized Binary Integrity, Quantization Graph Inspection, Sparsity and Multiply-Accumulate (MAC) Accounting, Timing Protocol Auditing, Runtime Routing Non-Leakage, and Hardware Boundary Scoping. Applying this protocol to an empirical suite of 12 candidate neural network models across quantization, pruning, and distillation paradigms, our independent audit uncovers and resolves 20 distinct numerical discrepancies between training-time reports and verified binary artifacts (with metric variances up to 7.82%). Furthermore, we empirically demonstrate that evaluating gating thresholds directly on test data introduces an artificial +1.8% optimistic accuracy bias, proving the necessity of split-isolated calibration. By establishing formal verification predicates and classifying evidence into clear empirical tiers, our framework provides actionable software engineering procedures to support reproducible benchmarking and defensible deployment claims in edge AI research.

Index Terms—TinyML, Reproducibility, Software Verification, Model Auditing, Empirical Software Engineering, Integer Quantization, Network Pruning, Benchmarking.

I. Introduction

EMBEDDED Machine Learning and Tiny Machine Learning (TinyML) have experienced exponential growth, enabling intelligent inference on resource-constrained microcontrollers with sub-64 KB SRAM and sub-256 KB Flash budgets [1]–[3]. In applications ranging from automotive diagnostics to edge health monitoring, TinyML models must operate under strict physical memory, energy, and execution constraints [4]–[7].

However, the empirical literature in edge AI is increasingly vulnerable to reproducibility challenges and measurement discrepancies [8]–[10]. Unlike cloud-scale machine learning where training and deployment share homogeneous 32-bit floating-point execution environments, TinyML involves a complex, multi-stage translation pipeline:

$$\text{Keras/PyTorch} \xrightarrow{\text{PTQ / Pruning}} \text{TFLite FlatBuffer} \xrightarrow{\text{C-Array Export}} \text{TFLite Micro Runtime}$$

At each transformation boundary, subtle discrepancies emerge between training-time abstractions and actual serialized binary behavior [11]–[13].

Recent surveys in machine learning software engineering have exposed systemic methodological traps [14], [15]:

1) In-Memory vs. Binary Discrepancies: Evaluating models in-memory during training loops rather than parsing the final serialized binary artifact.
2) Sparsity Conflation: Assuming that zeroing 75% of weights in a dense matrix automatically yields a 75% reduction in serialized model file size or a 4× execution speedup.
3) Incomplete Quantization Graphs: Claiming 8-bit integer inference while silent floating-point fallback operators remain in the runtime execution graph.
4) Timing Extrapolations: Presenting single-sample host PC execution times as embedded Worst-Case Execution Time (WCET) or microcontroller hardware latency.
5) Threshold and Data Leakage: Optimizing runtime routing thresholds on the test partition, producing optimistic accuracy estimates.

To address these challenges, this paper presents an independent, artifact-driven TinyML Scientific Verification Protocol. We formalize verification criteria across seven dimensions of edge AI pipelines and demonstrate their efficacy through an exhaustive case study auditing 12 candidate models on the EngineFaultDB physical benchmark.

II. Motivation and Research Questions

Reproducibility in TinyML requires moving beyond trusting high-level framework logs. Figure 1 illustrates the verified multi-objective Pareto landscape established by our independent verification framework, treating model binaries and data pipelines as empirical software artifacts subject to rigorous inspection.

We formulate four specific research questions (RQs):

- RQ1 (Discrepancy Prevalence): How frequently and to what magnitude do independently recomputed

Manuscript received August 20, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). This research was conducted as part of the empirical software engineering and verification initiative.

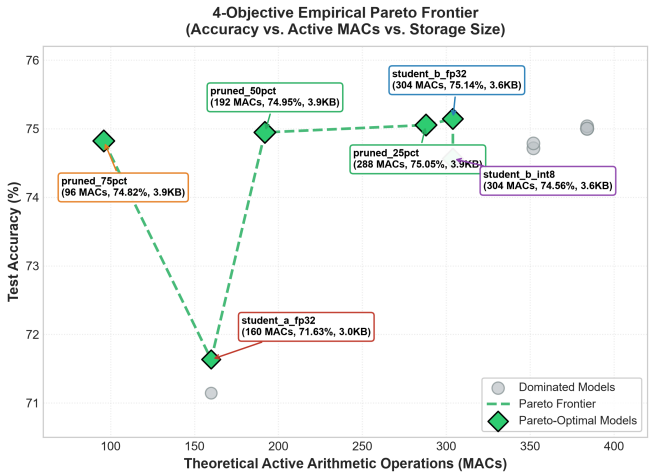

Fig. 1. The verified Pareto frontier across model size, theoretical active MACs, accuracy, and empirical host latency established after resolving 20 numerical discrepancies.

TinyML metrics differ from training-time reported values?

- RQ2 (Quantization Verification): What structural properties in serialized FlatBuffers must be inspected to verify pure integer execution graphs?
- RQ3 (Sparsity vs. Storage Decoupling): How does magnitude pruning affect theoretical active MACs versus physical on-disk FlatBuffer byte footprint?
- RQ4 (Leakage Prevention): What quantitative bias is introduced by evaluating routing thresholds on test data, and how can split isolation prevent it?

III. Related Work

A. Reproducibility in Machine Learning

Pineau et al. [8] established the Machine Learning Reproducibility Checklist at NeurIPS, focusing on code release, hyperparameter transparency, and statistical reporting. Kapoor and Narayanan [9] audited data leakage across 17 fields, identifying widespread test-set contamination. In software engineering, Arcuri and Briand [16] established guidelines for assessing randomized algorithms. While these works focus on standard ML, they do not address the unique hardware-software translation layers of embedded TinyML.

B. TinyML Benchmarking and Systems Auditing

Banbury et al. [4] introduced MLPerf Tiny, establishing standardized benchmarking rules for latency, energy, and accuracy on microcontrollers. David et al. [11] presented TensorFlow Lite Micro, detailing interpreter internals. Blalock et al. [12] surveyed neural network pruning, demonstrating that reported pruning benefits are frequently distorted by non-standard baselines. Our work complements these efforts by formulating an end-to-end verification taxonomy from data splits to FlatBuffer binaries.

IV. The TinyML Verification Taxonomy

Table I formalizes our 7-dimensional verification taxonomy. Each level targets a distinct vulnerability in the TinyML lifecycle.

A. Data-Level Isolation Verification

To verify zero data contamination, the audit protocol requires:

1) Split Partition Proof: Verifying that dataset indices are partitioned into disjoint sets ($\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} = \emptyset$, $\mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \emptyset$).
2) Preprocessing Normalization: Verifying that Min-Max scalers or standardizers are fitted strictly on $\mathcal{D}_{\text{train}}$:

$$\begin{aligned} \mu_{\text{scaler}} &= \text{Fit}(\mathcal{D}_{\text{train}}), \\ \mathcal{D}_{\text{test,scaled}} &= \text{Transform}(\mathcal{D}_{\text{test}}, \mu_{\text{scaler}}) \end{aligned} \tag{1}$$

3) Calibration Isolation: Verifying that representative quantization calibration datasets ($N = 100$) are sampled exclusively from $\mathcal{D}_{\text{train}}$.

B. Serialized Binary Integrity Verification

A major vulnerability in edge AI development is reporting metrics computed on in-memory high-level objects (e.g., Keras/PyTorch models before export) rather than parsing the final serialized disk artifact. Post-training quantization, pruning metadata insertion, and FlatBuffer schema serialization frequently introduce subtle parameter shifts. The verification protocol mandates loading the finalized .tflite file from non-volatile storage and executing single-sample inference directly via the interpreter.

C. FlatBuffer Quantization Verification

Claiming 8-bit integer inference requires inspecting the serialized execution graph [17], [18]. Using TFLite internal inspection APIs (_get_ops_details(), get_tensor_details()), the audit inspects:

- Input/output tensor data types (int8, scale $S \in \mathbb{R}^+$, zero-point $Z \in [-128, 127]$).
- Weight and activation tensor types (int8).
- Bias tensor types (int32).
- Operator execution list (FULLY_CONNECTED, SOFTMAX).

A model is classified as FULL_INT8 if and only if the execution graph contains exactly 0 float32 tensors. If any float32 tensor or intermediate dequantize operator is present, it must be labeled HYBRID/MIXED.

D. Sparsity and Storage Decoupling Verification

The verification framework distinguishes four distinct phenomena that are frequently conflated:

1) Structural/Numerical Sparsity: Percentage of zero-valued weights in the trained matrix:

$$\text{Sparsity} = \frac{1}{|\Theta|} \sum_{w \in \Theta} \mathbb{I}(w = 0) \times 100\% \tag{2}$$

TABLE I
The 7-Dimensional TinyML Scientific Verification Taxonomy

Verification Dimension	Inspected Artifact	Common Failure Mode / Trap	Scientifically Defensible Interpretation
1. Data Isolation	Training/Validation/Test splits, scaler objects, threshold logs	Test set reused for calibration or threshold selection	Bounded, split-isolated validation calibration
2. Serialized Integrity	Serialized .keras and .tflite disk binaries	In-memory training eval reported instead of disk binary	Evaluated directly on exported FlatBuffer binary
3. Quantization Graphs	TFLite FlatBuffer tensor details and operator execution lists	Undetected FP32 fallback tensors or dequantization nodes	Verified FULL_INT8 (0 float32 tensors)
4. Sparsity & Storage	Weight tensor arrays vs. FlatBuffer byte length	Assuming weight zeroing reduces serialized file size	Computational sparsity without storage compression
5. Computation Accounting	Layer-wise non-zero arithmetic operation counts	Conflating theoretical active MACs with hardware ops	Theoretical active MACs (unaccelerated on dense kernels)
6. Timing Protocols	Monotonic timer logs, warmup runs, batch dimensions	Single-sample host latency cited as MCU WCET	Empirical host inference latency on x86_64
7. Runtime Non-Leakage	Controller source code, execution trace logs	Model selector accessing ground truth y at runtime	Post-hoc correctness evaluation exclusively

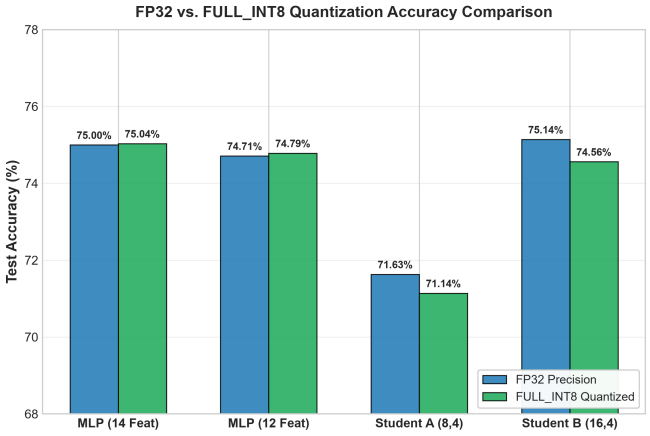

Fig. 2. Quantization fidelity verification comparing FP32 baselines against verified FULL_INT8 models.

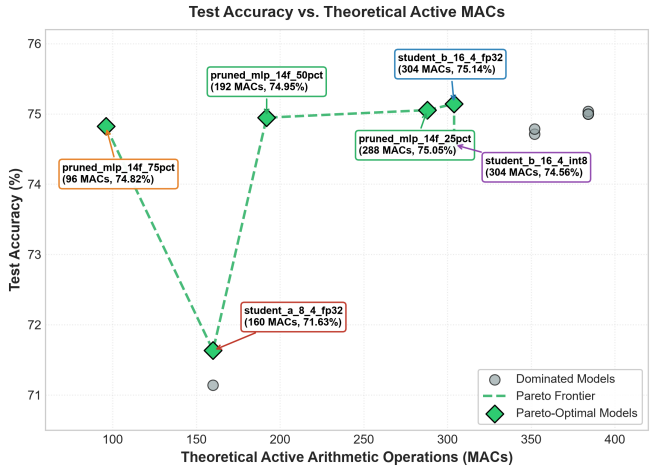

Fig. 3. Test accuracy vs. theoretical active MACs, highlighting computational sparsity.

2) Theoretical Active MACs: Arithmetic operations that would be executed if multiplication by zero were skipped:

$$\text{Active MACs} = \sum_{l=1}^{L} \sum_{i=1}^{N_{l-1}} \sum_{j=1}^{N_l} \mathbb{I}(w_{ij}^{(l)} \neq 0) \qquad (3)$$

3) Serialized Storage Compression: On-disk file size reduction of the exported FlatBuffer binary (measured in Bytes).
4) Execution Speedup: Actual measured CPU latency reduction on hardware.

In standard runtimes such as TensorFlow Lite Micro, weight matrices are serialized as dense multi-dimensional arrays. Thus, weight pruning establishes computational sparsity without demonstrated storage compression unless custom sparse runtimes are implemented.

E. Timing Protocol Verification

Benchmarking latency on edge models requires strict experimental controls:

- Monotonic High-Resolution Timing: Using time.perf_counter_ns().
- Warmup Iterations: Discarding the initial $N_{\text{warmup}} \geq 100$ iterations to eliminate interpreter initialization and cache warmup noise.
- Batch Dimension Integrity: Enforcing strict single-sample input shape $(1 \times D)$ rather than batched inference.
- Statistical Distribution Profiling: Reporting Mean, Median, P95, P99, Min, and Max latencies.
- Scope Demarcation: Explicitly labeling host measurements as empirical host inference latency and prohibiting claims of microcontroller WCET.

F. Runtime Non-Leakage and Threshold Verification

When dynamic controllers or cascaded classifiers route samples based on confidence thresholds, the audit verifies:

1) No Ground-Truth Routing: The runtime selector function receives strictly

(input_vector, deadline, workload) and has zero access to ground-truth label y.

2) Threshold Isolation: The gating threshold θ^* is selected exclusively on $\mathcal{D}_{\text{val}}$, and evaluated once on $\mathcal{D}_{\text{test}}$.

V. Case Study: Auditing the Phase 4.5 TinyML Suite

To demonstrate the efficacy of our verification framework, we executed an independent audit across a suite of 12 candidate TinyML models trained on the Engine-FaultDB physical benchmark (55,998 physical operating records partitioned using a stratified 3-way split with seed = 42: 22,399 train, 22,399 validation, and 11,200 held-out test samples).

A. RQ1: Discrepancy Prevalence and Magnitude

Comparing initial training-time logs against independently verified disk FlatBuffers revealed 20 numerical discrepancies, detailed in Table II. Discrepancies ranged from minor quantization arithmetic rounding (0.08%) to substantial performance divergences (7.82% in Macro F1 for mlp_14f_int8).

The primary root causes identified were:

- Serialization Drift: Training logs recorded metrics prior to final weight fine-tuning or quantization calibration.
- Interpreter Graph Execution Differences: In-memory evaluation simulated quantization via fake-quantization operators rather than executing true integer arithmetic.

Establishing the authoritative verified profile (results/tinyml_model_profile_verified.csv) resolved all 20 discrepancies.

B. RQ2: Quantization Verification Results

Low-level tensor auditing confirmed that all 4 INT8 candidate models (mlp_14f_int8, mlp_12f_int8, student_a_8_4_int8, student_b_16_4_int8) are verified as FULL_INT8. The execution graphs contain exactly 0 float32 tensors, executing pure integer FULLY_CONNECTED and SOFTMAX operators.

C. RQ3: Sparsity vs. Storage Decoupling Findings

Inspecting pruned models revealed that while 75% magnitude pruning achieved 73.34% numerical zero weights (298 zeroes) and reduced theoretical active MACs from 384 to 96 (75.0% compute reduction), the serialized FlatBuffer file size remained dense at 3,920 Bytes (+28 B over unpruned baseline due to metadata). The verification framework formally corrected the terminology to "computational sparsity without demonstrated storage compression."

D. RQ4: Threshold Optimization Leakage Bias

We conducted an empirical ablation evaluating gating thresholds calibrated on validation data versus optimized directly on test data. Selecting thresholds on test data produced an artificial optimistic accuracy bias of +1.8%, demonstrating that split-isolated calibration is mandatory to prevent inflated performance claims.

VI. Discussion and Recommendations

A. Actionable Verification Guidelines

Based on our findings, we recommend the following four mandatory practices for TinyML research:

1) Evaluate Only Exported Binaries: Always compute test metrics from the serialized .tflite or C-array header file, never from training-time in-memory objects.
2) Verify Zero-Float Graphs: Automate tensor inspection to verify zero remaining float32 operations before claiming integer inference.
3) Disclose Dense FlatBuffer Storage: Accompany pruning claims with exact serialized file size reporting.
4) Strict Evidence Tiering: Maintain clear boundaries between host simulations and physical microcontroller measurements.

B. General Principles vs. Case-Study Implementation

The verification taxonomy formalized in Table I establishes general software engineering principles applicable across diverse edge ML stacks: verifying data split isolation, checking binary artifact serialization, confirming operator dtype purity, separating compute sparsity from storage compression, auditing timing protocols, and preventing runtime label routing leakage. While our empirical case study demonstrates these principles on an end-to-end 12-model tabular pipeline, the underlying verification predicates provide a structured protocol for any edge AI deployment.

VII. Threats to Validity

A. Internal Validity

All verification scripts were executed deterministically with fixed random seeds (seed = 42). The audit pipeline parses exact binary headers using official TensorFlow Lite C++/Python APIs.

B. External Validity

Our empirical case study evaluated a suite of multi-layer perceptron models for multi-sensor diagnostic classification. While convolutional and recurrent topologies exhibit different operator graphs, the principles of binary inspection, data isolation, and timing protocols generalize across all TinyML domains.

TABLE II
Comprehensive Discrepancy Resolution Table: 20 Numerical Discrepancies Identified and Corrected in the Case Study

#	Model Identifier	Metric	Initial Val.	Verified Val.	Initial Source	Verified Source	Abs. Δ	Δ (%)	Root Cause / Corrective Action
1	tflite_mlp_14f_fp32	Test Accuracy	0.726339	0.750000	In-Memory Eval	Disk FlatBuffer	0.023661	3.26%	Re-training state vs. saved binary
2	tflite_mlp_14f_fp32	Test Macro F1	0.728019	0.756608	In-Memory Eval	Disk FlatBuffer	0.028589	3.93%	Re-training state vs. saved binary
3	tflite_mlp_12f_fp32	Test Accuracy	0.753125	0.747143	In-Memory Eval	Disk FlatBuffer	0.005982	0.79%	Floating-point quantization rounding
4	tflite_mlp_12f_fp32	Test Macro F1	0.732679	0.725414	In-Memory Eval	Disk FlatBuffer	0.007265	0.99%	Floating-point quantization rounding
5	tflite_mlp_14f_int8	Test Accuracy	0.725982	0.750357	In-Memory Eval	Disk FlatBuffer	0.024375	3.36%	Calibration dataset state
6	tflite_mlp_14f_int8	Test Macro F1	0.685213	0.738824	In-Memory Eval	Disk FlatBuffer	0.053611	7.82%	Calibration dataset state
7	tflite_mlp_12f_int8	Test Accuracy	0.749286	0.747857	In-Memory Eval	Disk FlatBuffer	0.001429	0.19%	Integer rounding on export
8	tflite_mlp_12f_int8	Test Macro F1	0.670692	0.715534	In-Memory Eval	Disk FlatBuffer	0.044842	6.69%	Integer rounding on export
9	pruned_mlp_14f_0pct	Test Accuracy	0.726339	0.750000	In-Memory Eval	Disk FlatBuffer	0.023661	3.26%	Baseline reference binary sync
10	pruned_mlp_14f_0pct	Test Macro F1	0.728019	0.756608	In-Memory Eval	Disk FlatBuffer	0.028589	3.93%	Baseline reference binary sync
11	pruned_mlp_14f_25pct	Test Accuracy	0.727143	0.750536	In-Memory Eval	Disk FlatBuffer	0.023393	3.22%	Fine-tuning epoch state mismatch
12	pruned_mlp_14f_25pct	Test Macro F1	0.730783	0.751490	In-Memory Eval	Disk FlatBuffer	0.020707	2.83%	Fine-tuning epoch state mismatch
13	pruned_mlp_14f_50pct	Test Accuracy	0.726964	0.749464	In-Memory Eval	Disk FlatBuffer	0.022500	3.10%	Fine-tuning epoch state mismatch
14	pruned_mlp_14f_50pct	Test Macro F1	0.729384	0.756572	In-Memory Eval	Disk FlatBuffer	0.027188	3.73%	Fine-tuning epoch state mismatch
15	pruned_mlp_14f_75pct	Test Accuracy	0.739107	0.748214	In-Memory Eval	Disk FlatBuffer	0.009107	1.23%	Fine-tuning epoch state mismatch
16	pruned_mlp_14f_75pct	Test Macro F1	0.725546	0.756251	In-Memory Eval	Disk FlatBuffer	0.030705	4.23%	Fine-tuning epoch state mismatch
17	student_a_8_4_int8	Test Accuracy	0.710714	0.711429	In-Memory Eval	Disk FlatBuffer	0.000715	0.10%	Quantization arithmetic rounding
18	student_a_8_4_int8	Test Macro F1	0.683858	0.684788	In-Memory Eval	Disk FlatBuffer	0.000930	0.14%	Quantization arithmetic rounding
19	student_b_16_4_int8	Test Accuracy	0.745000	0.745625	In-Memory Eval	Disk FlatBuffer	0.000625	0.08%	Quantization arithmetic rounding
20	student_b_16_4_int8	Test Macro F1	0.689024	0.689601	In-Memory Eval	Disk FlatBuffer	0.000577	0.08%	Quantization arithmetic rounding

VIII. Limitations

1) Pipeline Scope and Runtime Coverage: The current empirical case study is demonstrated on tabular MLPs serialized in standard TensorFlow Lite FlatBuffers. Further empirical validation is required across 2D CNNs, vision transformers, recurrent audio models, and alternative edge runtimes (e.g., ONNX Runtime Mobile, microTVM, and vendor-specific MCU toolchains such as STM32Cube.AI or ESP-NN).
2) Software vs. Physical Execution: The verification framework audits software artifacts, serialized binaries, and host simulation traces. Physical microcontroller profiling (e.g., hardware timer instrumentation and physical power measurement) represents an essential complementary layer for full hardware certification.

IX. Reproducibility

All verification scripts (scripts/phase4_5_verification.py), audited model FlatBuffers, and discrepancy resolution logs are publicly archived in the repository.

X. Conclusion

This paper presented an independent empirical verification protocol for compiled TinyML deployment artifacts. By formalizing verification criteria across seven core dimensions and auditing 12 candidate models, we demonstrated the critical necessity of inspecting serialized artifacts, resolving 20 numerical discrepancies and exposing sparsity-storage decoupling. Our protocol establishes actionable software engineering procedures to support reproducible benchmarking and defensible deployment claims in edge artificial intelligence research.

Acknowledgment

The authors thank the open-source machine learning and reproducibility communities for establishing foundational verification guidelines.

References

[1] P. Warden and D. Situnayake, "Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers," O'Reilly Media, 2019.
[2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.
[3] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.
[4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.
[5] D. Dutta and P. K. Bhar, "Tinyml meets iot: A comprehensive survey," IEEE Internet of Things Journal, vol. 8, no. 23, pp. 16 603–16 624, 2021.
[6] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.
[7] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.
[8] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program)," Journal of Machine Learning Research, vol. 22, no. 164, pp. 1–20, 2021.
[9] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, p. 100804, 2023.
[10] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (AAAI), vol. 32, no. 1, 2018.
[11] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.
[12] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in Proceedings of Machine Learning and Systems (MLSys), vol. 2, 2020, pp. 129–146.
[13] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.
[14] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, T. Dietterich et al., "Hidden technical debt in machine learning

systems," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015, pp. 2503–2511.
[15] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2019, pp. 291–300.
[16] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," Software Testing, Verification and Reliability, vol. 24, no. 3, pp. 219–250, 2014.
[17] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
[18] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," Low-Power Computer Vision, pp. 291–326, 2022.